

Security Insider Lab I

Lab 2: Machine Learning for Network Attack Detection

CIC-IDS2017 Friday DDoS Dataset

Group 8

Justlyn Kaniyanthara Johnson	121024
Aayush Kapoor	120957

---

9. November 2026

Contents

1 Introduction **1**

2 Task 1: Network Flows with CICFlowMeter **1**
2.1 What a Flow Is . . . 1
2.2 Capture Setup . . . 1
2.3 The IPv6 Problem . . . 1
2.4 The Final Capture . . . 2

3 Task 2: The CIC-IDS2017 Dataset **4**

4 Task 3: Data Pre-processing **4**
4.1 Results . . . 5
4.2 Interpretation . . . 8

5 Task 4: Machine Learning Classifiers **9**
5.1 Setup . . . 9
5.2 Results . . . 9
5.3 Interpretation . . . 9

6 Task 4: Real-Time Classification **11**
6.1 Setup . . . 11
6.2 Correctness: Parity Test . . . 11
6.3 Results on Our Capture . . . 12
6.4 Latency and Scope . . . 12

7 Discussion and Conclusion **13**

Bibliography **14**

Eidesstattliche Erklärung **15**

Abstract

In the second lab of the Security Insider Lab, we detect network attacks from flow features with machine learning. We first recorded our own traffic and converted it into flows with CICFlowMeter. Our first captures were mostly IPv6 and produced bogus flows, because CICFlowMeter force-parses non-IPv4 frames as IPv4. With IPv6 disabled, we obtained 569 flows, 547 of them valid TCP/UDP flows. As training data we use the Friday-afternoon DDoS file of CIC-IDS2017, which has 225,745 flows, 128,027 of them DDoS. Pre-processing removed infinite values, duplicate rows and constant and identical columns, which left 223,082 flows with 59 features. We then made a stratified split, min-max scaled the features and selected the 20 best with χ^2. We trained Logistic Regression and Decision Tree classifiers for the binary target BENIGN vs. DDoS. The default Decision Tree is best: on the test split of 66,925 flows it makes 7 false positives and 25 false negatives (DDoS F1 0.9996), whereas Logistic Regression raises about 2,000 false alarms (DDoS F1 0.974). Class weighting changes almost nothing, because DDoS is the majority class. Applied flow by flow to our own benign capture, the Decision Tree classified all 547 valid flows as BENIGN, so it raised no false alarm. Since the capture contains no attack, we cannot measure the detection rate there, and the near-perfect benchmark scores may partly rest on shortcut features such as the destination port.

List of Figures

2.1 CICFlowMeter in offline mode converting the capture, under its original file name `benign_lab2_3.pcap`, with the default timeouts (120 s flow, 5 s activity): 81,110 packets read (81,109 valid, 1 discarded), the same packet count as `task1_lab2.pcapng`. The reported "Total 570 flows" counts the lines of the output CSV including its header, i.e. 569 flows. 2

2.2 Disabling IPv6 on the capturing laptop before the third capture; `sysctl` confirms `net.ipv6.conf.all.disable_ipv6 = 1`. 2

2.3 Protocol hierarchy of `task1_lab2.pcapng` in Wireshark: 99.94 % of the frames are IPv4, only 24 are IPv6. 3

2.4 First rows of `task2_lab2.pcap_Flow.csv` in LibreOffice Calc: flow ID, endpoints, protocol, timestamp and the first features. (The `Label` column further to the right reads `NeedManualLabel`.) 3

4.1 Raw label summary as printed by the pre-processing script: 97,718 BENIGN and 128,027 DDoS flows (cf. Table 3.1). 5

4.2 Cleaning steps as logged by the pre-processing script, ending with "Final: 223082 rows x 59 features" (cf. Table 4.1). 6

4.3 Final label summary after cleaning and the class counts of the stratified 70/30 split (cf. Tables 3.1 and 4.3). 7

4.4 The top features by χ^2 as printed by the pre-processing script (binary target, ranks 1–20; cf. Table 4.5). 8

4.5 Top-20 flow features by χ^2 score for binary classification (BENIGN vs. DDoS), computed on the min-max-scaled training set. 8

5.1 Confusion matrices of two of the four models (test split). 10

5.2 Console output of the training script for the chosen model, `binary_dt_default`: per-class report, confusion matrix, and fit and inference times. 10
5.3 Comparison table of the four models, as printed by the training script (cf. Table 5.1). 11

6.1 Real-time architecture. Flows are streamed row by row in timestamp order. 11
6.2 Test evidence for Tasks 3 and 4. 12
6.3 Summary of the real-time detector: 569 flows read, 22 Protocol-0 rows dropped, 547 classified, 0 DDoS alerts, mean latency 13.537 ms per flow. 12

List of Tables

2.1 The three captures. Only the third, with IPv6 disabled, yields mostly valid flows. . . . 2
2.2 How non-IPv4 frames turn into fake IP addresses in the CICFlowMeter output. 3
2.3 Flows of `task2_lab2.pcap_Flow.csv`. The 22 flows with protocol 0 are not TCP/UDP flows and are dropped before classification. 4

3.1 Label distribution of the Friday file, raw and after the cleaning steps of Chapter 4. . . 4

4.1 Cleaning steps. The column counts include the `Label` column, so the 60 remaining columns are 59 features plus `Label`. Loading and label normalization happen before step 1. 6
4.2 Removed columns. For identical pairs, the first column is kept and the second one dropped. 6
4.3 Stratified 70/30 split (`random_state=42`). 6
4.4 Six example features of the training set before and after min-max scaling. 7
4.5 The 20 best features by χ^2 score (min-max-scaled training set), binary target. 7

5.1 Binary results on the test split (Prec., Rec., F1 refer to the DDoS class; Macro-F1 averages both classes; FP/FN are false positives and false negatives). Best model in bold. 10

6.1 Real-time run on `task2_lab2.pcap_Flow.csv`. 12

Listings

4.1 Core cleaning steps of `clean()` in `src/ddosids/data.py` (steps 2–6; logging calls omitted) 5

5.1 `build_pipeline()` and `make_classifier()` in `src/ddosids/models.py` 9

6.1 Excerpt of `Detector.process()` in `src/ddosids/realtime/detector.py`: column mapping, Protocol-0 drop, feature check, inf/NaN replacement and prediction (some lines omitted) 11

1 Introduction

In the second lab of the Security Insider Lab, we detect network attacks with machine learning. Instead of inspecting packets one by one, we describe each network connection by a vector of statistical features (a *flow*) and train classifiers to tell benign flows from attack flows. The lab has four tasks:

1. **Task 1:** we capture our own traffic and convert it into flows with CICFlowMeter.
2. **Task 2:** we analyze a labeled flow dataset, CIC-IDS2017.
3. **Task 3:** we pre-process the dataset: we handle infinite and missing values, scale the features, and select the 20 most informative ones.
4. **Task 4:** we train Logistic Regression and Decision Tree classifiers, evaluate them, and apply the best model to the flows of our own capture.

Dataset. As suggested in the lab slides, we use the Friday-afternoon DDoS file of CIC-IDS2017, `Friday-WorkingHours-Afternoon-DDos.csv`. It contains benign flows and flows of a distributed denial-of-service attack, so the task is a binary classification problem, BENIGN vs. DDoS.

Structure. Chapter 2 describes how we produced flows from our own capture and the problems we ran into. Chapter 3 introduces the dataset, Chapter 4 the pre-processing, and Chapter 5 the classifiers and their evaluation. Chapter 6 applies the best model to our own flows, and Chapter 7 discusses the results and limitations. All code is in our `ddosids` Python package; the listings in this report are excerpts of the real source files.

2 Task 1: Network Flows with CICFlowMeter

2.1 What a Flow Is

A flow summarizes a bidirectional exchange of packets between two endpoints, identified by source and destination IP address, ports and protocol. CICFlowMeter [Las+17; CFM] reads a packet capture and writes one row per flow with 84 columns: identifying metadata (flow ID, addresses, ports, protocol, timestamp), 77 numeric features (packet counts and lengths, inter-arrival times, TCP flag counts, window sizes, idle and active times) and a label column. A flow ends when a FIN or RST is seen, or when a timeout expires: 5 s without activity (activity timeout) or 120 s of total flow duration (flow timeout). CICFlowMeter only writes flows with at least two packets, so single-packet exchanges, such as a lone DNS query or a traceroute probe, do not appear in the output.

2.2 Capture Setup

We recorded our own traffic with Wireshark while browsing, resolving names and running the Steam client. The final capture, `task1_lab2.pcapng`, has 81,110 frames over 347.4 s (2026-11-04, 16:05–16:10). We converted it offline with CICFlowMeter to `task2_lab2.pcap_Flow.csv`. This file is the real-time input of Chapter 6. Since the traffic is ordinary browsing, we treat all flows as benign by construction.

2.3 The IPv6 Problem

Our first two captures produced CSV files in which most flows were garbage: protocol 0, port 0 and implausible IP addresses. Table 2.1 compares the captures. In the first two, 89–98 % of the frames were IPv6. After we disabled IPv6 on the capturing laptop for the third capture with `sudo sysctl -w net.ipv6.conf.all.disable_ipv6=1` (Figure 2.2), only 24 IPv6 frames (0.03 %) remained and the share of valid flows rose from 15 % and 5 % to 96 %.

Root cause. We traced the problem in the CICFlowMeter source. IPv6 support is switched off in the code: `readIP6 = false` is hard-coded in `Cmd.java` (line 126) and `ReadPcapFileWorker.java` (line 152). Every frame that is not IPv4 (IPv6, ARP, HomePlug powerline frames) falls through to `getVPNInfo()`,

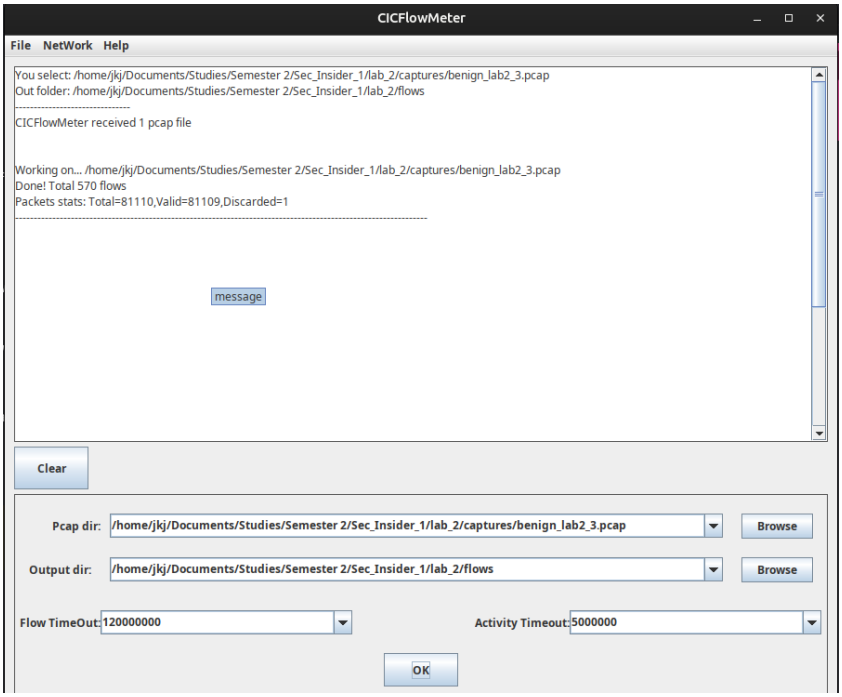


Figure 2.1: CICFlowMeter in offline mode converting the capture, under its original file name `benign_lab2_3.pcap`, with the default timeouts (120 s flow, 5 s activity): 81,110 packets read (81,109 valid, 1 discarded), the same packet count as `task1_lab2.pcapng`. The reported "Total 570 flows" counts the lines of the output CSV including its header, i.e. 569 flows.

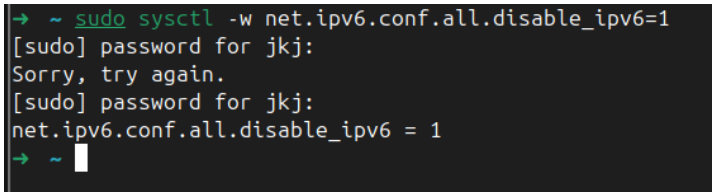


Figure 2.2: Disabling IPv6 on the capturing laptop before the third capture; `sysctl` confirms `net.ipv6.conf.all.disable_ipv6 = 1`.

Capture	Frames	IPv6 frames	Flows	Protocol 0	Valid TCP/UDP
1st (since deleted)	129,548	126,904 (98 %)	532	454	78
2nd (since deleted)	98,780	88,458 (89.5 %)	1,327	1,260	67
3rd, IPv6 off (final)	81,110	24 (0.03 %)	569	22	547

Table 2.1: The three captures. Only the third, with IPv6 disabled, yields mostly valid flows.

which force-parses the raw frame as an IPv4 header starting at byte 0. The fake "source IP" is therefore just the EtherType plus two further bytes, and the fake "destination IP" is taken from the IPv6 flow label and payload length (Table 2.2). The IPv4 flows themselves are parsed correctly. We decided against enabling CICFlowMeter's IPv6 path, because its UDP branch wrongly reads the header length from the TCP header (`PacketReader.java`, line 204). With IPv6 off, our flows are IPv4-only, like the training data.

2.4 The Final Capture

The final capture contains 47,682 UDP frames (29,682 QUIC, 580 DNS), 33,218 TCP frames (11,994 TLS, 26 HTTP), 160 ICMP frames and 26 ARP frames (Figure 2.3). Its offline conversion (Figure 2.1)

Fake “IP”	Bytes	Actually
`134.221.96.0`	`86 DD 60 00`	EtherType 0x86DD (IPv6) and first IPv6 header bytes
`0.0.4.214` (dst)	`00 00 04 D6`	IPv6 flow label and payload length (1238)
`136.225.0.0`	`88 E1 …`	EtherType 0x88E1 (HomePlug AV)
`8.6.x.x`	`08 06 …`	EtherType 0x0806 (ARP)

Table 2.2: How non-IPv4 frames turn into fake IP addresses in the CICFlowMeter output.

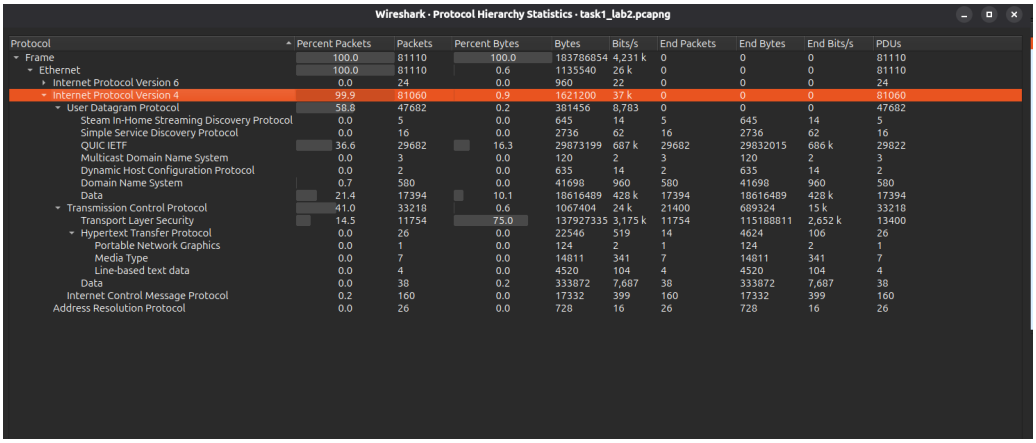
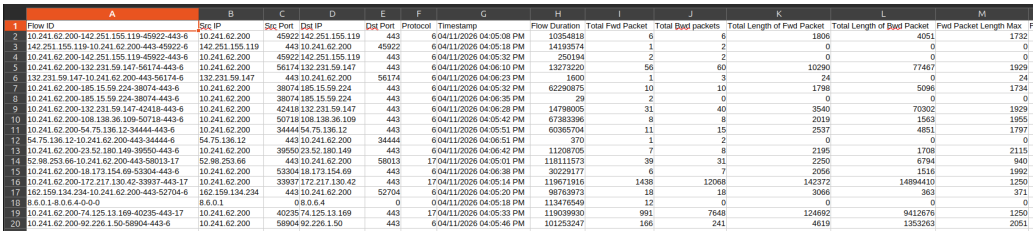


Figure 2.3: Protocol hierarchy of `task1_lab2.pcapng` in Wireshark: 99.94 % of the frames are IPv4, only 24 are IPv6.



Flow ID	Src IP	Src Port	Dst IP	Dst Port	Protocol	Timestamp	Flow Duration	Total Fwd Packet	Total Bwd packets	Total Length of Fwd Packet	Total Length of Bwd Packet	Fwd Packet Length Max
10.241.62.200-142.251.155.119-45922-443-6	10.241.62.200	45922	142.251.155.119	443	6	04/11/2026 04:05:08 PM	10354818	6	6	1806	4051	1732
142.251.155.119-10.241.62.200-443-45922-6	142.251.155.119	443	10.241.62.200	45922	6	04/11/2026 04:05:18 PM	14193574	1	2	0	0	0
10.241.62.200-142.251.155.119-45922-443-6	10.241.62.200	45922	142.251.155.119	443	6	04/11/2026 04:05:32 PM	250194	2	2	0	0	0
10.241.62.200-132.231.59.147-56174-443-6	10.241.62.200	56174	132.231.59.147	443	6	04/11/2026 04:06:10 PM	13273220	56	60	10290	77467	1929
132.231.59.147-10.241.62.200-443-56174-6	132.231.59.147	443	10.241.62.200	56174	6	04/11/2026 04:06:23 PM	1600	1	3	24	0	24
10.241.62.200-185.15.59.224-38074-443-6	10.241.62.200	38074	185.15.59.224	443	6	04/11/2026 04:05:32 PM	62290875	10	10	1798	5096	1734
10.241.62.200-185.15.59.224-38074-443-6	10.241.62.200	38074	185.15.59.224	443	6	04/11/2026 04:06:35 PM	29	2	0	0	0	0
10.241.62.200-132.231.59.147-42418-443-6	10.241.62.200	42418	132.231.59.147	443	6	04/11/2026 04:06:28 PM	14798005	31	40	3540	70302	1929
10.241.62.200-108.138.36.109-50718-443-6	10.241.62.200	50718	108.138.36.109	443	6	04/11/2026 04:05:42 PM	67383396	8	8	2019	1563	1955
10.241.62.200-54.75.136.12-34444-443-6	10.241.62.200	34444	54.75.136.12	443	6	04/11/2026 04:05:51 PM	60365704	11	15	2537	4851	1797
54.75.136.12-10.241.62.200-443-34444-6	54.75.136.12	443	10.241.62.200	34444	6	04/11/2026 04:06:51 PM	370	1	2	0	0	0
10.241.62.200-23.52.180.149-39550-443-6	10.241.62.200	39550	23.52.180.149	443	6	04/11/2026 04:06:42 PM	11208705	7	8	2195	1708	2115
52.98.253.66-10.241.62.200-443-58013-17	52.98.253.66	443	10.241.62.200	58013	17	04/11/2026 04:05:01 PM	118111573	39	31	2250	6794	940
10.241.62.200-18.173.154.69-53304-443-6	10.241.62.200	53304	18.173.154.69	443	6	04/11/2026 04:06:38 PM	30229177	6	7	2056	1516	1992
10.241.62.200-172.217.130.42-33937-443-17	10.241.62.200	33937	172.217.130.42	443	17	04/11/2026 04:05:14 PM	119671916	1438	12068	142372	14894410	1250
162.159.134.234-10.241.62.200-443-52704-6	162.159.134.234	443	10.241.62.200	52704	6	04/11/2026 04:05:20 PM	98763973	18	18	3066	363	371
8.6.0.1-8.0.6.4-0-0-0	8.6.0.1	0	8.0.6.4	0	0	04/11/2026 04:05:18 PM	113476549	12	0	0	0	0
10.241.62.200-74.125.13.169-40235-443-17	10.241.62.200	40235	74.125.13.169	443	17	04/11/2026 04:05:33 PM	119039930	991	7648	124692	9412676	1250
10.241.62.200-92.226.1.50-58904-443-6	10.241.62.200	58904	92.226.1.50	443	6	04/11/2026 04:05:46 PM	101253247	166	241	4619	1353263	2051

Figure 2.4: First rows of `task2_lab2.pcap_Flow.csv` in LibreOffice Calc: flow ID, endpoints, protocol, timestamp and the first features. (The `Label` column further to the right reads `NeedManualLabel`.)

yields 569 flows (Table 2.3, Figure 2.4). More than half of the classified flows (289) are DNS queries to the local resolver, since every query is its own flow; 213 go to port 443 and 15 to port 80. The classified flows contain 80,705 packets, which is 99.8 % of the 80,900 IPv4 TCP/UDP frames. The missing packets are single-packet UDP exchanges, mostly traceroute probes, which CICFlowMeter does not write.

The 22 protocol-0 flows consist of 15 real ICMP flows (CICFlowMeter only sets the protocol number for TCP and UDP), 4 flows from left-over IPv6 frames and 3 from ARP frames, the last two groups mis-parsed as described above. The real-time detector drops all 22 (3.9 %) and logs the count. There are no infinite or NaN values in the remaining flows, and all 77 features map to the training feature names (Section 6.1). About 70 % of the valid flows are UDP (DNS, QUIC and Steam). The application mix (e.g. QUIC and Steam) and the network itself differ from the 2017 testbed, so we expect domain shift (Chapter 6).

Quantity	Value
Total flows	569
TCP flows (6)	162
UDP flows (17)	385
Protocol 0 flows (dropped)	22
Flows classified (TCP + UDP)	547
Packets in classified flows	80 705
Top-1 destination port	53 (289 flows)
Top-2 destination port	443 (213 flows)
Top-3 destination port	80 (15 flows)

Table 2.3: Flows of `task2_lab2.pcap_Flow.csv`. The 22 flows with protocol 0 are not TCP/UDP flows and are dropped before classification.

3 Task 2: The CIC-IDS2017 Dataset

CIC-IDS2017 was created by the Canadian Institute for Cybersecurity at the University of New Brunswick [SLG18; CIC]. The flows were extracted with CICFlowMeter [Las+17]. We use the pre-computed "MachineLearningCSV" file `Friday-WorkingHours-Afternoon-DDos.csv` (77 MB). It has 225,745 flows and 79 columns: 78 numeric flow features and the `Label`. It contains no flow ID, IP address or timestamp columns, but `Destination Port` is present. The file covers the DDoS attack of the Friday afternoon, as described in the dataset paper.

The file has two labels, BENIGN and DDoS, and the attack is the *majority* class: DDoS makes up 56.71 % of the raw flows (Table 3.1). The classes are therefore reasonably balanced, and accuracy is less misleading than on a dataset with a rare attack. We still report per-class precision, recall and F1, the macro-F1 and confusion matrices throughout, never accuracy alone. Virtually all DDoS flows target destination port 80, while benign traffic is spread over several ports (mostly 53, 443 and 80). Models can exploit this as a shortcut (Section 4.2 and Chapter 7).

Label	Raw	Share	Cleaned	Share
BENIGN	97 718	43.29 %	95 068	42.62 %
DDoS	128 027	56.71 %	128 014	57.38 %
Total	**225 745**	**100.00 %**	**223 082**	**100.00 %**

Table 3.1: Label distribution of the Friday file, raw and after the cleaning steps of Chapter 4.

4 Task 3: Data Pre-processing

The task asks us to analyze the labels, handle infinite and NaN values, scale the features with min-max scaling and select the 20 best features with `SelectKBest`. We implemented this in Python 3.12 with pandas and scikit-learn [Ped+11] (modules `data.py` and `features.py`, script `task3_preprocess.py`). The steps are, in this order:

1. **Load and analyze labels.** 65 of the 79 column names carry stray leading spaces and are stripped. The labels are normalized (whitespace collapsed, validated against the two expected classes). For this file this step changes nothing, because the labels are already clean.
2. **Drop the duplicate column.** `Fwd Header Length` appears twice in the header, a known artifact of the CICFlowMeter export. We drop the second copy.
3. **Infinity and NaN.** We replace $\pm\infty$ by NaN and *drop* the affected rows. Only 34 rows are affected, 32 BENIGN and 2 DDoS, so dropping loses almost no information and introduces no artificial values. The infinite and NaN values occur only in `Flow Bytes/s` and `Flow Packets/s`: CICFlowMeter divides by the flow duration, which is 0 for some flows (a positive byte count

divided by 0 gives ∞, 0/0 gives NaN).

4. **Duplicate rows.** We remove exact duplicates before splitting. Otherwise identical rows land in both train and test sets and inflate the test scores.
5. **Constant columns.** They carry no information and trigger warnings in `SelectKBest`.
6. **Identical columns.** Columns equal to an earlier one (tolerance 10^{-6}) are removed, so that the 20 selected features are distinct. The tolerance is needed for one pair, `Bwd Packet Length Mean` and `Avg Bwd Segment Size`: they are the same quantity but differ in 2 rows by at most 10^{-6} because of CSV rounding.
7. **Train/test split.** A stratified 70/30 split on the label with `random_state=42`, done *before* anything is fitted.
8. **Min-max scaling.** `MinMaxScaler(clip=True)` maps each feature to $[0, 1]$. It is fitted on the training set only, so no test statistics leak into training, and `clip=True` keeps unseen values, such as live traffic later, inside $[0, 1]$.
9. **Feature selection.** `SelectKBest(chi2, k=20)` on the scaled training set. We scale first because χ^2 requires non-negative input, and the raw data contains negative values: `Init_Win_bytes_*` is -1 when no window was observed, and a few durations are negative.

The column-level steps 5 and 6 use feature values only, never labels, so they leak no label information. Listing 4.1 shows the core of steps 2–6 in the code.

Listing 4.1: Core cleaning steps of `clean()` in `src/ddosids/data.py` (steps 2–6; logging calls omitted)

```
    if DUP_HEADER_COL in df.columns:
        df = df.drop(columns=[DUP_HEADER_COL])
    feats = [c for c in df.columns if c != LABEL_COL]
    num = df[feats]
    inf_counts = {c: int(v) for c, v in np.isinf(num).sum().items() if v}
    df[feats] = num.replace([np.inf, -np.inf], np.nan)
    mask = df[feats].isna().any(axis=1)
    bd = _breakdown(df.loc[mask, LABEL_COL])
    df = df.loc[~mask]
    dup = df.duplicated()
    bd = _breakdown(df.loc[dup, LABEL_COL])
    df = df.loc[~dup]
    feats = [c for c in df.columns if c != LABEL_COL]
    const = [c for c in feats if df[c].nunique(dropna=False) <= 1]
    df = df.drop(columns=const)
    feats = [c for c in df.columns if c != LABEL_COL]
    pairs = _identical_columns(df[feats])
    df = df.drop(columns=[b for _, b in pairs])
```

4.1 Results

```
======================================================================
Raw label summary
======================================================================
        count  percent  ratio_to_benign
Label
BENIGN  97718  43.2869         1.000000
DDoS   128027  56.7131         1.310168
```

Figure 4.1: Raw label summary as printed by the pre-processing script: 97,718 BENIGN and 128,027 DDoS flows (cf. Table 3.1).

Table 4.1 lists the cleaning steps with their effect (Figure 4.2 shows the script log), and Table 4.2 the removed columns. The raw file has 2,633 duplicate rows, but 4 of them contain infinite or NaN values and are already gone after step 3, so step 4 drops 2,629 (2,618 BENIGN, 11 DDoS). The six bulk-transfer features are always 0 in this CICFlowMeter version. `Fwd PSH Flags` is identical to `SYN Flag Count`, although PSH and SYN are different TCP flags. This points to a feature-extraction problem in the CICFlowMeter version used for CIC-IDS2017, in line with the literature [ERJ21].

#	Step	Rows	Cols	Detail
1	Drop duplicate header column	225 745→225 745	79→78	−1 column (`Fwd Header Length.1`)
2	Replace ±∞ by NaN	225 745→225 745	78→78	inf: `Flow Bytes/s` 30, `Flow Packets/s` 34; NaN: `Flow Bytes/s` 4
3	Drop NaN rows	225 745→225 711	78→78	−34 rows (32 BENIGN, 2 DDoS)
4	Drop duplicate rows	225 711→223 082	78→78	−2 629 rows (2 618 BENIGN, 11 DDoS)
5	Drop constant columns	223 082→223 082	78→68	−10 columns
6	Drop identical columns	223 082→223 082	68→60	−8 columns
	Result	**223 082**		**223 082 rows × 59 features**

Table 4.1: Cleaning steps. The column counts include the `Label` column, so the 60 remaining columns are 59 features plus `Label`. Loading and label normalization happen before step 1.

```
================================================================
Cleaning steps
================================================================
- drop_duplicate_header_column: rows 225745 -> 225745, cols 79 -> 78
    dropped: ['Fwd Header Length.1']
- replace_inf_with_nan: rows 225745 -> 225745, cols 78 -> 78
    inf_per_column: {'Flow Bytes/s': 30, 'Flow Packets/s': 34}
    nan_per_column: {'Flow Bytes/s': 4}
- drop_nan_rows: rows 225745 -> 225711, cols 78 -> 78
    dropped_rows: 34
    dropped_by_label: {'BENIGN': 32, 'DDoS': 2}
- drop_duplicate_rows: rows 225711 -> 223082, cols 78 -> 78
    dropped_rows: 2629
    dropped_by_label: {'BENIGN': 2618, 'DDoS': 11}
- drop_constant_columns: rows 223082 -> 223082, cols 78 -> 68
    dropped: ['Bwd PSH Flags', 'Fwd URG Flags', 'Bwd URG Flags', 'CWE Flag Count', 'Fwd Avg Bytes/Bulk', 'Fwd Avg Packets/Bulk', 'Fwd Avg Bulk Rate', 'Bwd Avg Bytes/Bulk', 'Bwd Avg Packets/Bulk', 'Bwd Avg Bulk Rate']
- drop_identical_columns: rows 223082 -> 223082, cols 68 -> 60
    dropped: ['SYN Flag Count', 'ECE Flag Count', 'Avg Fwd Segment Size', 'Avg Bwd Segment Size', 'Subflow Fwd Packets', 'Subflow Fwd Bytes', 'Subflow Bwd Packets', 'Subflow Bwd Bytes']
    pairs: [['Fwd PSH Flags', 'SYN Flag Count'], ['RST Flag Count', 'ECE Flag Count'], ['Fwd Packet Length Mean', 'Avg Fwd Segment Size'], ['Bwd Packet Length Mean', 'Avg Bwd Segment Size'], ['Total Fwd Packets', 'Subflow Fwd Packets'], ['Total Length of Fwd Packets', 'Subflow Fwd Bytes'], ['Total Backward Packets', 'Subflow Bwd Packets'], ['Total Length of Bwd Packets', 'Subflow Bwd Bytes']]
    atol: 1e-06

Final: 223082 rows x 59 features (+ Label, binary_label)
```

Figure 4.2: Cleaning steps as logged by the pre-processing script, ending with "Final: 223082 rows x 59 features" (cf. Table 4.1).

Kind	Columns
Constant (10)	`Bwd PSH Flags`, `Fwd URG Flags`, `Bwd URG Flags`, `CWE Flag Count`, `Fwd Avg Bytes/Bulk`, `Fwd Avg Packets/Bulk`, `Fwd Avg Bulk Rate`, `Bwd Avg Bytes/Bulk`, `Bwd Avg Packets/Bulk`, `Bwd Avg Bulk Rate`
Identical	`Fwd PSH Flags` → `SYN Flag Count` (dropped)
Identical	`RST Flag Count` → `ECE Flag Count` (dropped)
Identical	`Fwd Packet Length Mean` → `Avg Fwd Segment Size` (dropped)
Identical	`Bwd Packet Length Mean` → `Avg Bwd Segment Size` (dropped)
Identical	`Total Fwd Packets` → `Subflow Fwd Packets` (dropped)
Identical	`Total Length of Fwd Packets` → `Subflow Fwd Bytes` (dropped)
Identical	`Total Backward Packets` → `Subflow Bwd Packets` (dropped)
Identical	`Total Length of Bwd Packets` → `Subflow Bwd Bytes` (dropped)

Table 4.2: Removed columns. For identical pairs, the first column is kept and the second one dropped.

After cleaning, 223,082 flows with 59 features remain: 95,068 BENIGN and 128,014 DDoS (Table 3.1), so DDoS stays the majority class. Table 4.3 shows the split: the test set contains 66,925 flows, of which 38,404 are DDoS.

Label	Train (70 %)	Test (30 %)
BENIGN	66 547	28 521
DDoS	89 610	38 404
Total	**156 157**	**66 925**

Table 4.3: Stratified 70/30 split (`random_state=42`).

On the training set, each of the six example features in Table 4.4 has a scaled minimum of 0 and a maximum of 1. The durations and inter-arrival times, with maxima near the 120 s timeout, are

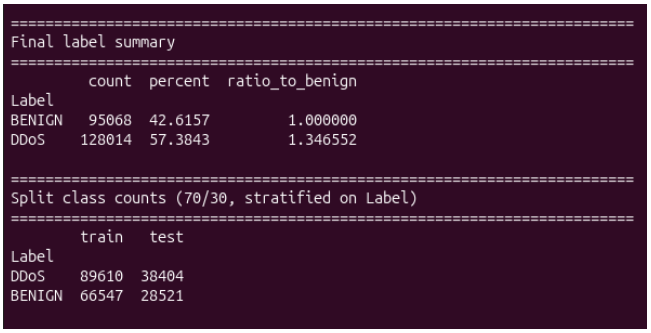


Figure 4.3: Final label summary after cleaning and the class counts of the stratified 70/30 split (cf. Tables 3.1 and 4.3).

heavy-tailed and keep modest means after scaling.

	Before scaling			After scaling		
Feature	Min	Max	Mean	Min	Max	Mean
`Init_Win_bytes_backward`	−1	65 535	616.5311	0	1	0.0094
`Init_Win_bytes_forward`	−1	65 535	4 289	0	1	0.0655
`Destination Port`	0	65 532	9 036	0	1	0.1379
`Flow Duration`	−1	119 999 937	16 472 365	0	1	0.1373
`Bwd IAT Total`	0	120 000 000	6 626 749	0	1	0.0552
`PSH Flag Count`	0	1	0.3556	0	1	0.3556

Table 4.4: Six example features of the training set before and after min-max scaling.

`URG Flag Count` and `Destination Port` lead the 20 selected features (Table 4.5; scores in Figure 4.5, script output in Figure 4.4) with scores above 28,000 each, and the scores fall steadily down the ranking.

Rank	Binary feature	χ^2
1	`URG Flag Count`	30058.11
2	`Destination Port`	28476.04
3	`Bwd Packet Length Max`	22196.57
4	`Bwd Packet Length Std`	15201.54
5	`Bwd Packet Length Mean`	13631.13
6	`Bwd IAT Total`	10104.71
7	`Max Packet Length`	9940.91
8	`Packet Length Mean`	9735.77
9	`Fwd Packet Length Max`	9338.26
10	`Packet Length Std`	9282.63
11	`Average Packet Size`	8361.12
12	`Fwd Packet Length Mean`	8326.59
13	`Fwd PSH Flags`	7104.49
14	`Packet Length Variance`	6816.96
15	`Bwd IAT Max`	6626.18
16	`Fwd Packet Length Std`	6519.34
17	`PSH Flag Count`	5771.95
18	`Down/Up Ratio`	4499.68
19	`Fwd Packet Length Min`	3409.00
20	`Bwd IAT Std`	3351.57

Table 4.5: The 20 best features by χ^2 score (min-max-scaled training set), binary target.

```
======================================================================
Top features by chi2: binary
======================================================================
 rank               feature     score  p_value
    1         URG Flag Count 3.006e+04        0
    2       Destination Port 2.848e+04        0
    3  Bwd Packet Length Max  2.22e+04        0
    4  Bwd Packet Length Std  1.52e+04        0
    5 Bwd Packet Length Mean 1.363e+04        0
    6          Bwd IAT Total  1.01e+04        0
    7      Max Packet Length      9941        0
    8     Packet Length Mean      9736        0
    9  Fwd Packet Length Max      9338        0
   10      Packet Length Std      9283        0
   11    Average Packet Size      8361        0
   12 Fwd Packet Length Mean      8327        0
   13          Fwd PSH Flags      7104        0
   14 Packet Length Variance      6817        0
   15            Bwd IAT Max      6626        0
   16  Fwd Packet Length Std      6519        0
   17         PSH Flag Count      5772        0
   18          Down/Up Ratio      4500        0
   19  Fwd Packet Length Min      3409        0
   20            Bwd IAT Std      3352        0
```

Figure 4.4: The top features by χ^2 as printed by the pre-processing script (binary target, ranks 1–20; cf. Table 4.5).

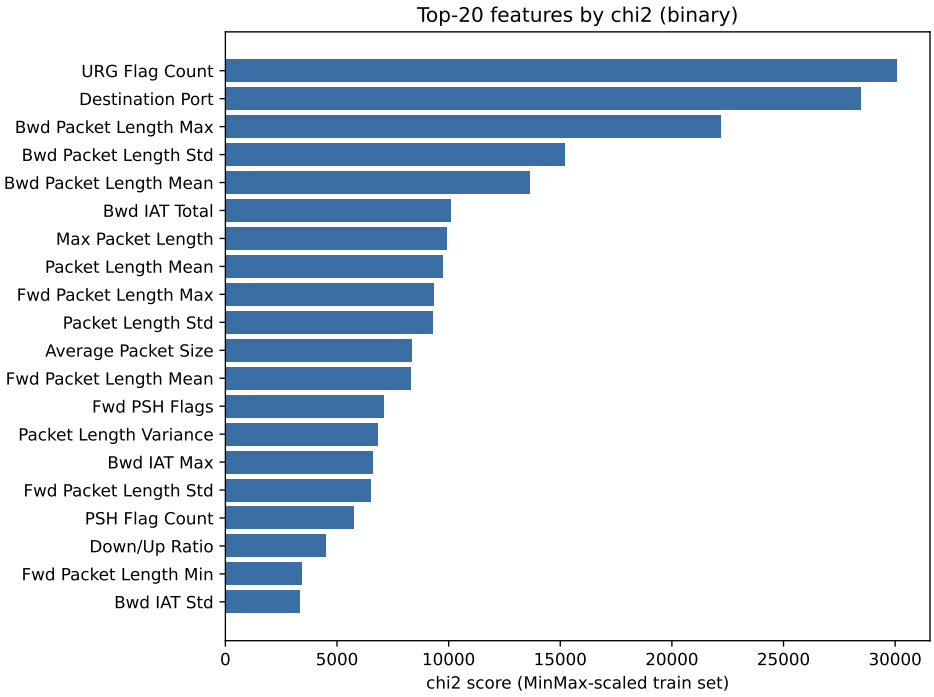


Figure 4.5: Top-20 flow features by χ^2 score for binary classification (BENIGN vs. DDoS), computed on the min-max-scaled training set.

4.2 Interpretation

The top-20 list is dominated by packet-size features: twelve of the 20 features describe packet lengths (forward and backward length statistics, the overall maximum, mean, standard deviation and variance, and the average packet size). This is plausible for a flooding attack, whose requests are likely more uniform than the mix of benign traffic. Three features are TCP flag counts (`URG Flag Count`, `Fwd PSH Flags`, `PSH Flag Count`), three describe backward inter-arrival times, and `Down/Up Ratio` captures the traffic asymmetry. `Destination Port` ranks second, and virtually all DDoS flows target port 80, so a model can learn "port 80 is suspicious" instead of the attack behavior. We expect such testbed-specific features to generalize poorly to other networks. The χ^2 test on min-max-scaled continuous data is also

only a heuristic: it treats scaled values like frequencies, and heavy-tailed features are squeezed towards 0, which lowers their scores. The ranking therefore depends on the scaling.

5 Task 4: Machine Learning Classifiers

5.1 Setup

We train Logistic Regression (LR, `max_iter=5000`) and a Decision Tree (DT, `random_state=42`, unpruned) with scikit-learn [Ped+11]. Each is trained in two variants, default and `class_weight="balanced"`. The file has only one attack class, so a multiclass target would duplicate the binary one (BENIGN vs. DDoS), and we train four binary models. Every model is one scikit-learn `Pipeline`: `MinMaxScaler(clip=True)`, `SelectKBest(chi2, k=20)` and the classifier (Listing 5.1). Scaler and selector are fitted on the training split only (156,157 flows). All models are evaluated on the held-out test split of 66,925 flows, of which 38,404 are DDoS. Inference time is the median of five timed `predict` calls on the whole test split after one warm-up call, scaled to 1,000 flows. Both LR models converged after 48 iterations, far below the limit of 5,000.

Listing 5.1: `build_pipeline()` and `make_classifier()` in `src/ddosids/models.py`

```python
def build_pipeline(clf: Any, k: int = K) -> Pipeline:
    return Pipeline([
        ("scale", MinMaxScaler(clip=True)),
        ("select", SelectKBest(chi2, k=k)),
        ("clf", clf),
    ])
def make_classifier(model: str) -> Any:
    makers = {
        "lr_default": lambda: LogisticRegression(max_iter=LR_MAX_ITER),
        "lr_balanced": lambda: LogisticRegression(max_iter=LR_MAX_ITER, class_weight="balanced"),
        "dt_default": lambda: DecisionTreeClassifier(random_state=RANDOM_STATE),
        "dt_balanced": lambda: DecisionTreeClassifier(
            random_state=RANDOM_STATE, class_weight="balanced"),
    }
    return makers[model]()
```

5.2 Results

The default DT makes only 7 false positives and 25 false negatives on 66,925 test flows (Table 5.1), which gives a DDoS precision of 0.9998, a recall of 0.9993 and an F1 of 0.9996. Logistic Regression detects almost as many attacks (recall 0.9988, 45 missed flows) but raises about 2,000 false alarms (2,008 for the default and 1,974 for the balanced variant), so its DDoS precision is only about 0.95 and its F1 0.974. Figure 5.1 shows the confusion matrices of the default DT and the balanced LR, and Figure 5.2 the console output for the default DT.

Class weighting changes almost nothing. For the DT, the balanced variant has exactly the same confusion matrix and metrics as the default one, and for LR the false positives drop only from 2,008 to 1,974. The two classes are close to balanced (57 % DDoS and 43 % BENIGN after cleaning), so `class_weight="balanced"` gives both classes weights close to one. The fit and inference times come from a single run each.

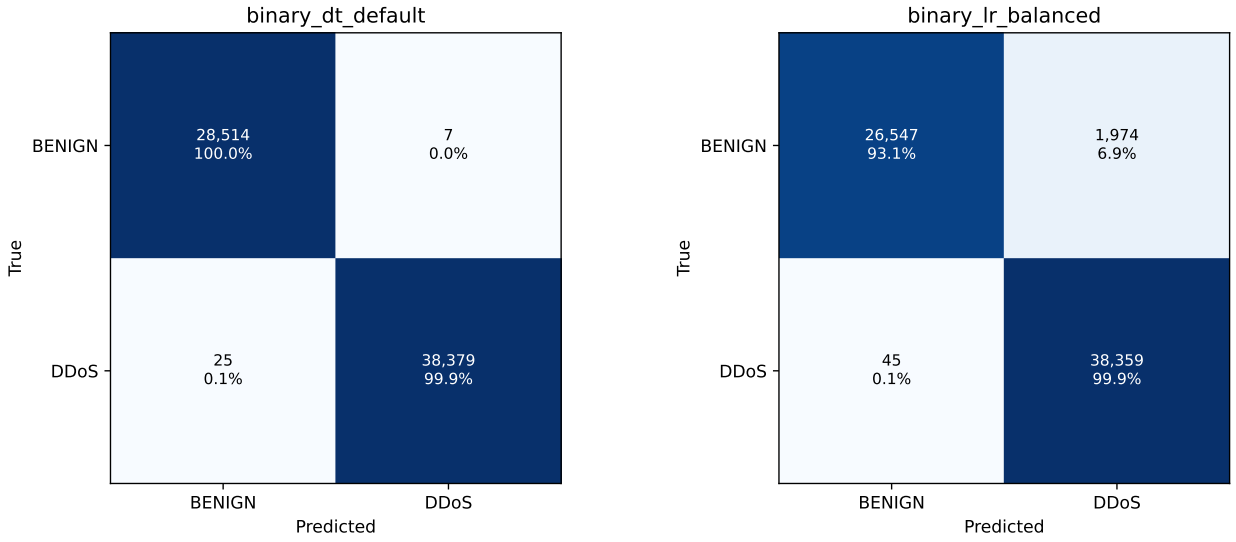
5.3 Interpretation

The Decision Tree beats Logistic Regression in every metric of Table 5.1: the DDoS F1 is 0.9996 against at most 0.9744. Attacks occupy narrow value ranges of a few features (packet lengths, port 80, flag counts), and a tree isolates such ranges with axis-aligned thresholds. LR draws a single linear boundary through heavy-tailed, min-max-compressed features and cannot separate the classes as sharply.

We use `binary_dt_default` in the real-time part. Default and balanced DT are identical on the test

Model	Acc.	Macro-F1	Prec.	Rec.	F1	FP	FN	Fit (s)	Inf. (ms/1k)
LR default	0.9693	0.9683	0.9503	0.9988	0.9739	2 008	45	6.45	0.717
LR balanced	0.9698	0.9689	0.9511	0.9988	0.9744	1 974	45	5.97	1.996
DT default	**0.9995**	**0.9995**	**0.9998**	**0.9993**	**0.9996**	**7**	**25**	**2.20**	**0.695**
DT balanced	0.9995	0.9995	0.9998	0.9993	0.9996	7	25	1.25	0.717

Table 5.1: Binary results on the test split (Prec., Rec., F1 refer to the DDoS class; Macro-F1 averages both classes; FP/FN are false positives and false negatives). Best model in bold.



(a) Decision Tree (default): 7 false positives, 25 false negatives.

(b) Logistic Regression (balanced): 1,974 false positives, 45 false negatives.

Figure 5.1: Confusion matrices of two of the four models (test split).

```
======================================================================
binary_dt_default
======================================================================
       class  precision  recall     f1  support
      BENIGN     0.9991  0.9998 0.9994    28521
        DDoS     0.9998  0.9993 0.9996    38404
   macro avg     0.9995  0.9996 0.9995    66925
weighted avg        NaN     NaN 0.9995    66925
    accuracy        NaN     NaN 0.9995    66925
accuracy = 0.9995 | macro-F1 = 0.9995 | weighted-F1 = 0.9995
Confusion matrix (rows = true, cols = predicted):
             pred BENIGN  pred DDoS
true BENIGN        28514          7
true DDoS             25      38379
fit = 2.20 s | inference = 0.695 ms per 1,000 flows
----------------------------------------------------------------------
```

Figure 5.2: Console output of the training script for the chosen model, `binary_dt_default`: per-class report, confusion matrix, and fit and inference times.

split, so we take the simpler default model, and it has the fewest errors of all four models. `Destination Port` is the second-best feature by χ^2 (Table 4.5), and virtually all DDoS flows target port 80, so the near-perfect scores may partly rest on the port as a shortcut (Chapter 7). Because the test split comes from the same capture as the training data, Chapter 6 also checks the model on traffic from a different network.

```
======================================================================
Comparison table
======================================================================
              key target       model  accuracy  macro_f1  weighted_f1  f1_BENIGN  f1_DDoS  ddos_precision  ddos_recall  fit_s  infer_ms_per_1k
 binary_lr_default binary  lr_default    0.9693    0.9683       0.9692     0.9627   0.9739          0.9503       0.9988 6.4513           0.7167
binary_lr_balanced binary lr_balanced    0.9698    0.9689       0.9697     0.9634   0.9744          0.9511       0.9988 5.9728           1.9960
 binary_dt_default binary  dt_default    0.9995    0.9995       0.9995     0.9994   0.9996          0.9998       0.9993 2.1979           0.6954
binary_dt_balanced binary dt_balanced    0.9995    0.9995       0.9995     0.9994   0.9996          0.9998       0.9993 1.2523           0.7165
```

Figure 5.3: Comparison table of the four models, as printed by the training script (cf. Table 5.1).

6 Task 4: Real-Time Classification

6.1 Setup

For the real-time part, we use the binary default Decision Tree (`binary_dt_default`). It has the best DDoS F1 and the fewest errors on the test split (7 false positives among 28,521 benign flows and 25 false negatives among 38,404 DDoS flows). The detector loads the saved pipeline unchanged, so training and serving use exactly the same scaling, selection and classifier. Figure 6.1 shows the data flow. The input is the flow CSV of our own capture (Chapter 2): 569 flows produced by CICFlowMeter V4, whereas the training data comes from an older version. The detector streams the flows one by one in timestamp order, maps the V4 column names to the training names (all 77 features map), drops the `Protocol = 0` rows, replaces infinite and NaN values by 0 and classifies each flow (Listing 6.1). The flow CSV is a command-line argument, so the detector can be pointed at any other V4 flow file.

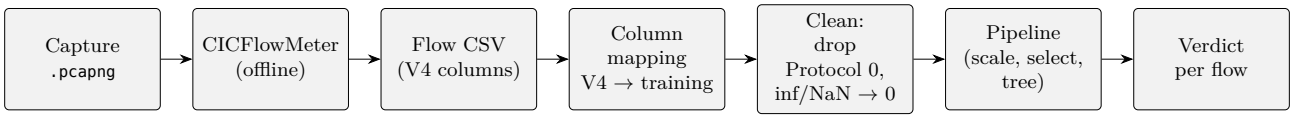


Figure 6.1: Real-time architecture. Flows are streamed row by row in timestamp order.

Listing 6.1: Excerpt of `Detector.process()` in `src/ddosids/realtime/detector.py`: column mapping, Protocol-0 drop, feature check, inf/NaN replacement and prediction (some lines omitted)

```
        df = map_v4_to_iscx(df_v4)
        # 2. drop Protocol-0 (non TCP/UDP / unparsed) rows
        n_proto0 = 0
        if "Protocol" in df.columns:
            keep = pd.to_numeric(df["Protocol"], errors="coerce").fillna(0) != 0
            n_proto0 = int((~keep).sum())
            df = df[keep]
        missing = [c for c in self.input_features if c not in df.columns]
        if missing:
            raise ValueError(f"missing required feature columns: {missing}")
        x = df[self.input_features].apply(pd.to_numeric, errors="coerce").astype(float)
        bad = ~np.isfinite(x.to_numpy())
        n_replaced = int(bad.sum())
        x = x.mask(bad, 0.0)
            pred = self.pipeline.predict(x)
            proba = self.pipeline.predict_proba(x)
            p_att = proba[:, self.classes.index(1)]
            labels = np.where(pred == 1, config.ATTACK_NAME, config.BENIGN)
            is_alert = np.asarray(pred == 1, dtype=bool)
```

6.2 Correctness: Parity Test

To prove that the real-time path does not change the model's behavior, an end-to-end test converts the 66,925-flow test split to the V4 column names and streams it through the detector. The streamed predictions are identical, row for row, to those of the offline pipeline, both for batches of 1,000 flows and, on the first 500 flows, for batches of one flow. The DDoS F1 recomputed from the streamed predictions equals the offline value of Table 5.1. Figure 6.2 shows the test evidence: 24 fast tests pass

(3 slow end-to-end tests deselected), and the 3 end-to-end tests (Task 3 numbers, Task 4 regression guards and this parity test) pass as well.

```
> uv run pytest -m "not e2e" -q
.......................
24 passed, 3 deselected in 6.25s
```

(a) Fast tests: 24 passed, 3 deselected.

```
> uv run pytest -m e2e -q
...
3 passed, 24 deselected in 51.13s
```

(b) End-to-end tests: 3 passed, 24 deselected.

Figure 6.2: Test evidence for Tasks 3 and 4.

6.3 Results on Our Capture

Of 569 flows, 22 were dropped (protocol 0) and 547 were classified, all of them as BENIGN: the detector raised 0 DDoS alerts (Table 6.1, Figure 6.3). No infinite or NaN cells had to be replaced. Our traffic is benign by construction, so every flow is correctly classified and there is no false positive.

Quantity	Value
Flows read	569
Protocol-0 flows dropped	22
inf/NaN cells replaced	0
Flows classified	547
Classified BENIGN	547
DDoS alerts	0
Alert rate	0.00 %
Latency per flow, mean (ms)	13.537
Latency per flow, median (ms)	12.840

Table 6.1: Real-time run on `task2_lab2.pcap_Flow.csv`.

```
================================================================
Summary
================================================================
model                    : binary_dt_default
flows read               : 569
Protocol-0 rows dropped  : 22
inf/NaN cells replaced   : 0
flows classified         : 547
BENIGN                   : 547
DDoS (alerts)            : 0
alert rate               : 0.00 %
latency per flow (ms)    : mean 13.537 | median 12.840
Alert flows: none
```

Figure 6.3: Summary of the real-time detector: 569 flows read, 22 Protocol-0 rows dropped, 547 classified, 0 DDoS alerts, mean latency 13.537 ms per flow.

6.4 Latency and Scope

Classifying one flow at a time takes 13.537 ms on average (median 12.840 ms) in our recorded run. This is per-call pandas and scikit-learn overhead: batched, the same model needs 0.695 ms per 1,000 flows (Table 5.1). In practice, the detection delay is dominated by CICFlowMeter, which emits a flow only when it ends (FIN or RST) or times out (5 s activity, 120 s flow timeout). "Real time" here means streaming the flows of a recorded capture through the detector one by one; we do not capture packets live.

Limitations of this check. Our capture contains only benign traffic, so we can measure the false-positive behavior on our network but not the detection rate: whether the model would flag a real

DDoS attack in our flows remains untested. In addition, there is domain shift. The training data was produced by an older CICFlowMeter version on the 2017 testbed, whereas our flows come from CICFlowMeter V4 on a different network with a different application mix (for example QUIC and Steam traffic). A model that relies on testbed-specific shortcut features, such as the destination port, may behave differently on other traffic, in either direction.

7 Discussion and Conclusion

Shortcut features and domain shift. The near-perfect benchmark scores (DDoS F1 0.9996) may partly rest on a shortcut. `Destination Port` ranks second among the selected features, and virtually all DDoS flows target port 80, so the model may have learned the port rather than the attack behavior (Section 4.2). The test split comes from the same capture as the training data, so it cannot reveal such a shortcut. The CICFlowMeter version, the hosts and the application mix also differ between the 2017 testbed and our network (Section 6.4). The 0 alerts on our capture show only that this particular benign traffic is not mistaken for DDoS. The data itself also has quality problems, such as the duplicated column, infinite values, 2,633 duplicate rows and several TCP flag columns that are identical or constant. They match issues reported in the literature [ERJ21], and each of them needed its own cleaning step in Chapter 4.

Class balance. Because the classes are close to balanced, class weighting has almost no effect: the balanced and default Decision Trees are identical on the test split, and the balanced Logistic Regression removes only 34 of about 2,000 false alarms.

Limitations. We use a single train/test split without cross-validation. The tree is unpruned. We ran no ablation without `Destination Port`, so we cannot quantify the shortcut. Dropping rows with infinite values removes 34 flows, including 2 DDoS flows, and min-max scaling is sensitive to outliers. Finally, the real-time part replays recorded flows and is not live capture.

Possible improvements (not implemented). Pruning the tree (`max_depth`, `min_samples_leaf`), dropping the testbed-specific features (such as `Destination Port`), evaluating on a second attack capture from another network, and adding benign traffic from the deployment network to the training data would make the conclusions more reliable.

Conclusion. We turned the Friday DDoS file of CIC-IDS2017 into a clean set of 223,082 flows with 59 features, selected the 20 best features, and trained Logistic Regression and Decision Tree classifiers for BENIGN vs. DDoS. The Decision Tree outperforms Logistic Regression and makes only 32 errors on 66,925 test flows. On our own benign traffic, it raised no false alarm, but because our capture has no attack and the data may allow shortcuts, its benchmark score does not by itself show how well it would detect DDoS attacks in our network.

Bibliography

[ERJ21] Gints Engelen, Vera Rimmer, and Wouter Joosen. "Troubleshooting an Intrusion Detection Dataset: the CICIDS2017 Case Study." In: *2021 IEEE Security and Privacy Workshops (SPW)*. 2021, pp. 7–12. DOI: 10.1109/SPW53761.2021.00009 (cit. on pp. 5, 13).

[CIC] *IDS 2017 Dataset*. Canadian Institute for Cybersecurity, University of New Brunswick. URL: https://www.unb.ca/cic/datasets/ids-2017.html (visited on 11/04/2026) (cit. on p. 4).

[CFM] Arash Habibi Lashkari. *CICFlowMeter*. URL: https://github.com/ahlashkari/CICFlowMeter (visited on 11/04/2026) (cit. on p. 1).

[Las+17] Arash Habibi Lashkari et al. "Characterization of Tor Traffic Using Time Based Features." In: *Proceedings of the 3rd International Conference on Information Systems Security and Privacy (ICISSP)*. 2017, pp. 253–262. DOI: 10.5220/0006105602530262 (cit. on pp. 1, 4).

[Ped+11] Fabian Pedregosa et al. "Scikit-learn: Machine Learning in Python." In: *Journal of Machine Learning Research* 12.85 (2011), pp. 2825–2830. URL: https://jmlr.org/papers/v12/pedregosa11a.html (cit. on pp. 4, 9).

[SLG18] Iman Sharafaldin, Arash Habibi Lashkari, and Ali A. Ghorbani. "Toward Generating a New Intrusion Detection Dataset and Intrusion Traffic Characterization." In: *Proceedings of the 4th International Conference on Information Systems Security and Privacy (ICISSP)*. 2018, pp. 108–116. DOI: 10.5220/0006639801080116 (cit. on p. 4).

Eidesstattliche Erklärung

Hiermit versichere ich, dass ich diese MasterLab selbstständig und ohne Benutzung anderer als der angegebenen Quellen und Hilfsmittel angefertigt habe und alle Ausführungen, die wörtlich oder sinngemäß übernommen wurden, als solche gekennzeichnet sind, sowie, dass ich die MasterLab in gleicher oder ähnlicher Form noch keiner anderen Prüfungsbehörde vorgelegt habe.

Passau, 9. November 2026

Justlyn Kaniyanthara Johnson

Aayush Kapoor